

Distributed Systems - 3_11

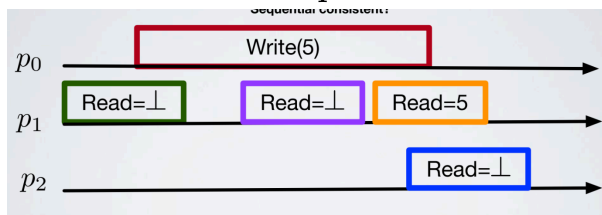
The definition of Atomicity is:

- In an execution, each operation should appear to take effect instantaneously at some moment between its start and completion.

So, for each operation, we should find a point in which the operation happens instantly. Before this point, known as linearisation point, the operation still didn't happen.

This linearisation point is equivalent to taking a lock.

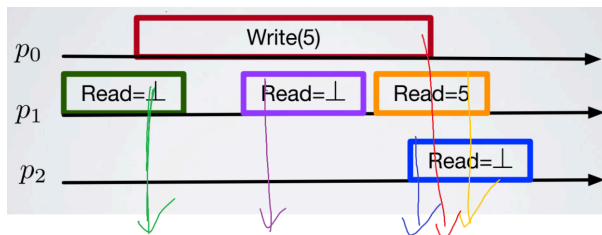
Is this execution seq. consistent?



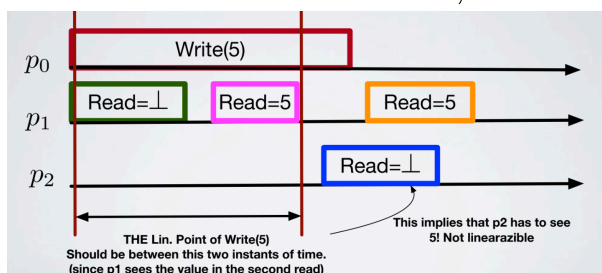
Yes, we could have green $\rightarrow$ purple $\rightarrow$ blue $\rightarrow$ red $\rightarrow$ yellow.

Is this regular? Yes, as once we are concurrent we can either read the new value or the old one.

Is this atomic? Yes, we could have:



Let's take another execution, is it linearizable?



No, as seen in photo the blue read gives a problem, as the linearization point of the read is necessarily after the linearization point of the write.

Though it's sequential consistent, as we could have blue $\rightarrow$ green $\rightarrow$ red $\rightarrow$ pink $\rightarrow$ yellow. It's also regular.

Can I find an execution that is atomic and not regular? No, as imagine that we could violate regular, which means that we have a write operation that writes a value v , and then an operation reads another value $v' \neq v$

Every run that is linearizable is also a regular one. So Atomicity $\implies$ Regular, but the opposite is not true.

Now, can we find an execution that is atomic, but not sequential consistent? If we have linearization points, we have an order between operations. So it's telling to us that we can order operations, to make the system behave well. So it's impossible to have atomicity and not sequential consistency.

So, Atomicity $\implies$ Sequential.

So, we have Atomicity $\implies$ Sequential Consistency and Atomicity $\implies$ Regular, but Regular $\not\implies$ Consistency.

Let's do an exercise:

Consider the execution depicted in the following figure and answer the questions

1. Define ALL the values that can be returned by read operations (Rx) assuming the run refers to a regular register.

2. Define ALL the values that can be returned by read operations (Rx) assuming the run refers to an atomic register.

To solve this, we can draw line "che coincidono" with the write operations. This helps us to understand which operations are concurrent.

So, we have the following:

$read_1$ is either 1 or

$read_2$ is either 1 or 2 or $\perp$

$read_3$ is either 2, or 3, or 1

$read_4$ return 2 or 3, as we are concurrent with 3, and 2 is the value that was previously on the register

$read_5$ either returns 2 or 3, as p_1 is dead and so operations are concurrent forever in Regular Consistency.

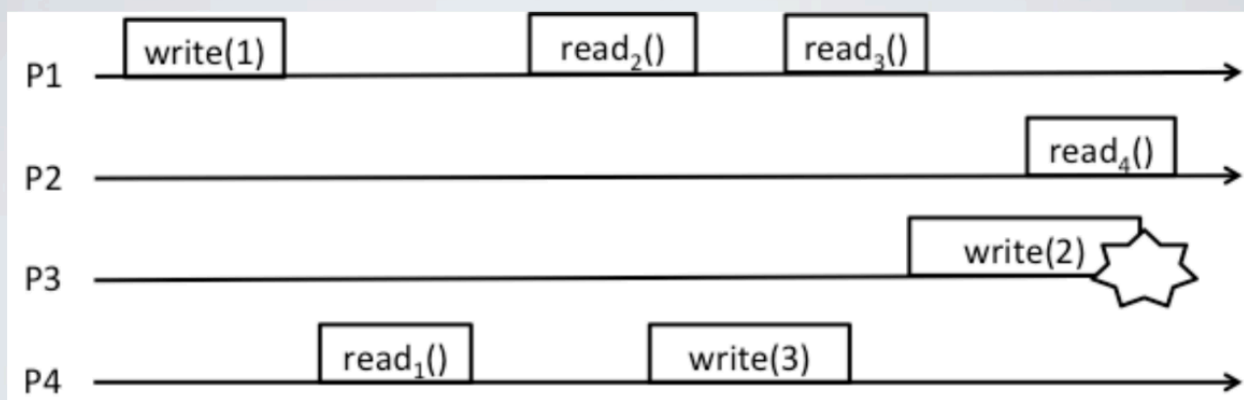
For the second point, we do the same but now the register is atomic.

Let's take, for example, read operations r_2 and r_3 . The value of r_2 depends on r_3 and viceversa. In fact if r_2 return 2, the the linearization point of the $w(2)$ would be before the linearization point of r_2 . Then, r_3 could return 2 if the linearization point of $w(3)$ was after the one of r_3 , otherwise 3. If r_2 returned 1, then r_3 could return either 1, 2 or 3.

So, to solve this kind of operations, we can define a tree structure for each case. Starting from r_2 , we would have different branches, one for r_2 outputting 2, one outputting 1 and one outputting $\perp$. Then, we see what happens with r_1 with r_2 outputting 2 (for example), and we could have r_1 outputting 1 or 2. And we proceed like this.

Let's do another exercise:

Consider the execution depicted in the following figure and answer the questions



1. Define ALL the values that can be returned by read operations (R_x) assuming the run refers to a regular register.
2. Define ALL the values that can be returned by read operations (R_x) assuming the run refers to an atomic register.

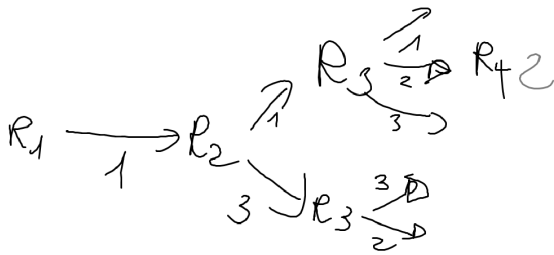
r_1 returns 1

r_2 returns 1,3

r_3 return 1,3,2

r_4 returns 3,2

For atomic, let's draw the tree:

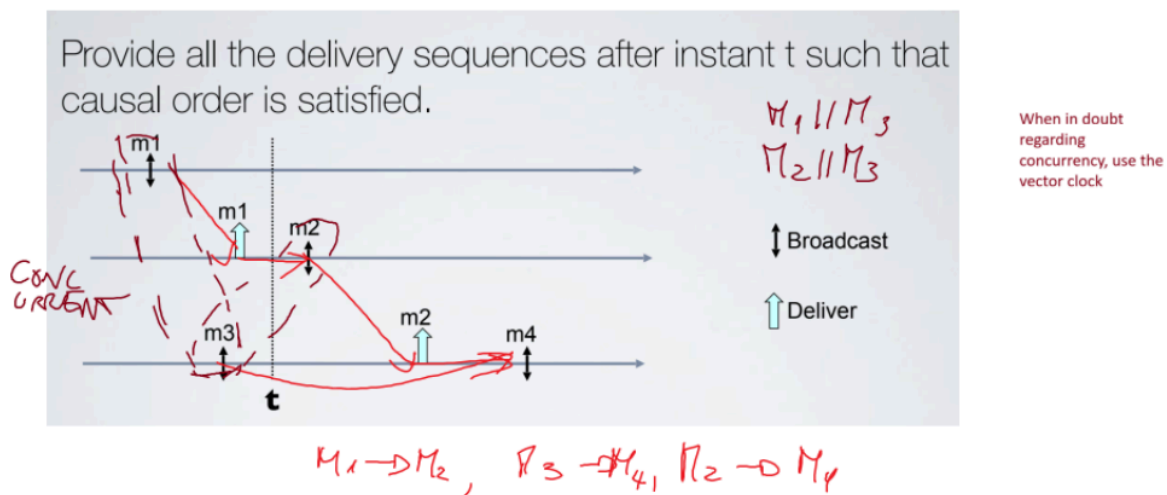


Let's do another exercise:

How do we solve this kind of exercises? We make assumption, assuming a message is delivered before a broadcast, having determined relationship between message, or after a broadcast (example of m_2 , we could deliver m_3 before the broadcast of m_2 or after).

In the example, we assume m_3 is delivered after the broadcast of m_2 .

Remember that we modify executions only after time t .



$P_0: m_1, m_2, m_3, m_4$ or m_1, m_3, m_2, m_4 or m_3, m_1, m_2, m_4
 $P_1: m_1, m_2, m_3, m_4$ or m_1, m_3, m_2, m_4
 $P_2: m_1, m_3, m_2, m_4$ or m_1, m_2, m_3, m_4

P.S., in p_2 we can also deliver m_1, m_3, m_2, m_4 .

Now, let's see how to implement the atomic register.

We will see atomic registers with a single writer and n readers.

If we have a single writer, then the definition of atomicity can be simplified.

We'll have the following properties:

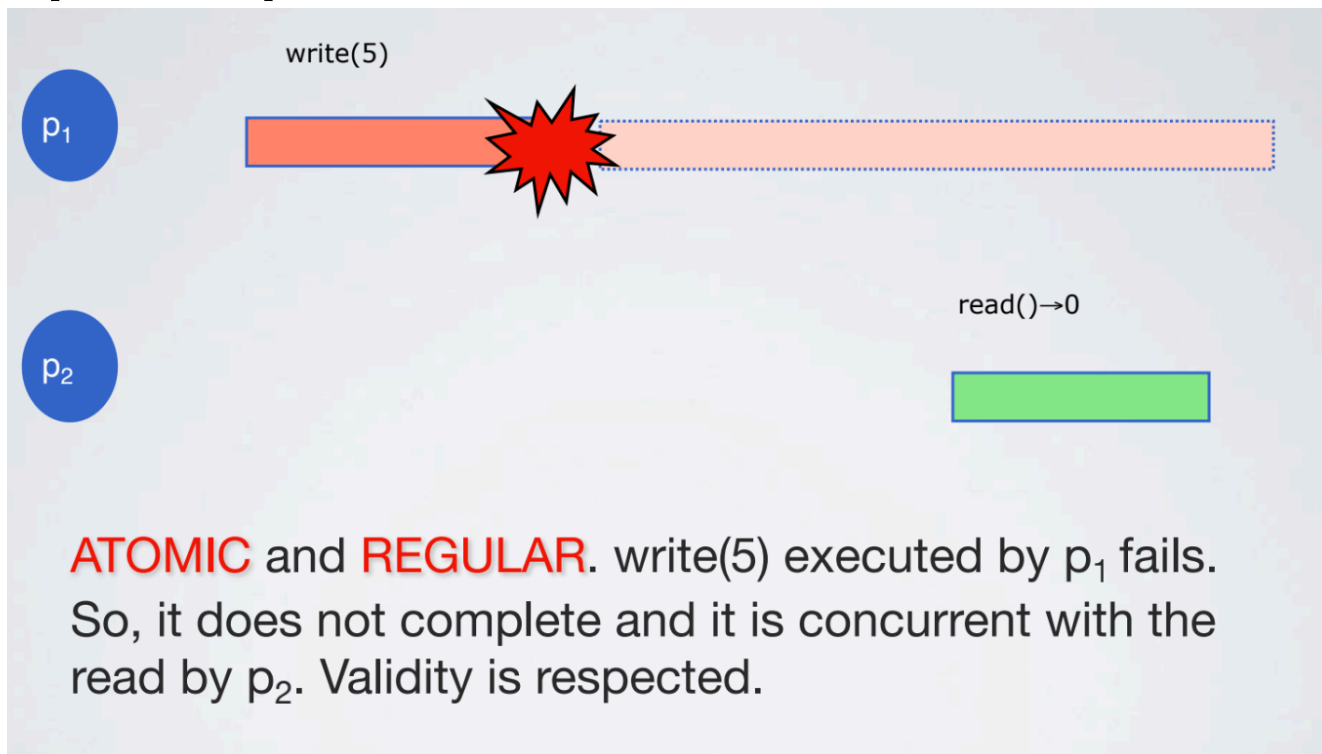
Termination. If a correct process invokes an operation, then the operation eventually receives the corresponding confirmation.

Validity. A read operation returns the last value written or the value concurrently being written.

Ordering. If a read returns v_2 after a read that precedes it has returned v_1 , then v_1 has not been written after v_2

(Ordering is stated for (1,N) and it is equivalent to the more general definition with Linearization Points).

Important example:



The interface is the same as the regular one, with the same events and properties, plus a new one:

- (Ordering): If a read returns a value v and a subsequent read returns a value w , then the write of w does not precede the write of v .

We will see how to implement the atomic in two ways:

1. Implementation using regular registers to communicate, without the use of "normal" messages (with "normal" I mean no messages except the ones we use to manage the registers)
2. Implementation via messages

Why we see two different ways to implement it? If we can solve a problem with messages, then we can solve it with shared memory, and this is true for the opposite as well.

We'll have two phases in the building:

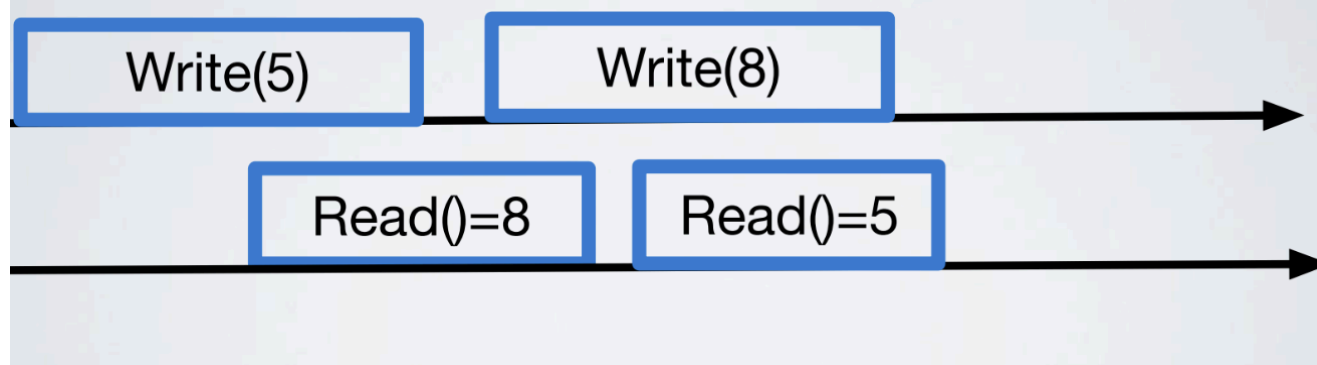
- Phase 1: We use a $(1, N)$ regular register to build a $(1, 1)$ atomic register
- Phase 2: We use a set of $(1, 1)$ atomic registers to build a $(1, N)$ atomic register.

We have to fix some notations: When we talk about RR , we are talking about the regular register, AR register is the atomic one.

So, in the first phase we'll build the $(1, 1)$ atomic register on top of the $(1, N)$ regular register.

What could give a problem in atomic registers starting from the regular register? If we have two reads concurrent with a write, like the first read reading 5, the second $\perp$, and the write overlaps part of the two writing 5.

IDEA 1: We have just one writer and one reader, what is the pathological case with a RR that is not acceptable in AR?



How can we fix this? We use timestamps, when the writer writes he will write $(5, 0)$, with the new value he will write $(8, 1)$. So, with the second read, we would read $(5, 0)$, but because the timestamp is smaller than the one in $(8, 1)$, then I send that one that I saved.

So we use a $(1, N)$ RR where p_1 is the writer and p_2 is the reader. Each write operation writes (val, ts) on the register. The reader tracks the timestamp of previously read value and discards old value.

Algorithm 4.3: From $(1, N)$ Regular to $(1, 1)$ Atomic Registers

```

(1, 1)-AtomicRegister, instance ooar.    upon event  $\langle onrr, WriteReturn \rangle$  do
                                              trigger  $\langle ooar, WriteReturn \rangle$ ;

```

$(1, N)$ -RegularRegister, **instance** *onrr*. **upon event** $\langle \text{oar}, \text{Read} \rangle$ **do**
trigger $\langle \text{onrr}, \text{Read} \rangle$;

$$\begin{aligned}(ts, val) &:= (0, \perp); \\ wts &:= 0;\end{aligned}$$
$$(ts, val) := (ts', v');$$
$$wts := wts + 1;$$

```
trigger < ooar, ReadReturn | val >;
```

```
trigger  $\langle onrr, Write \mid (wts, v) \rangle;$ 
```

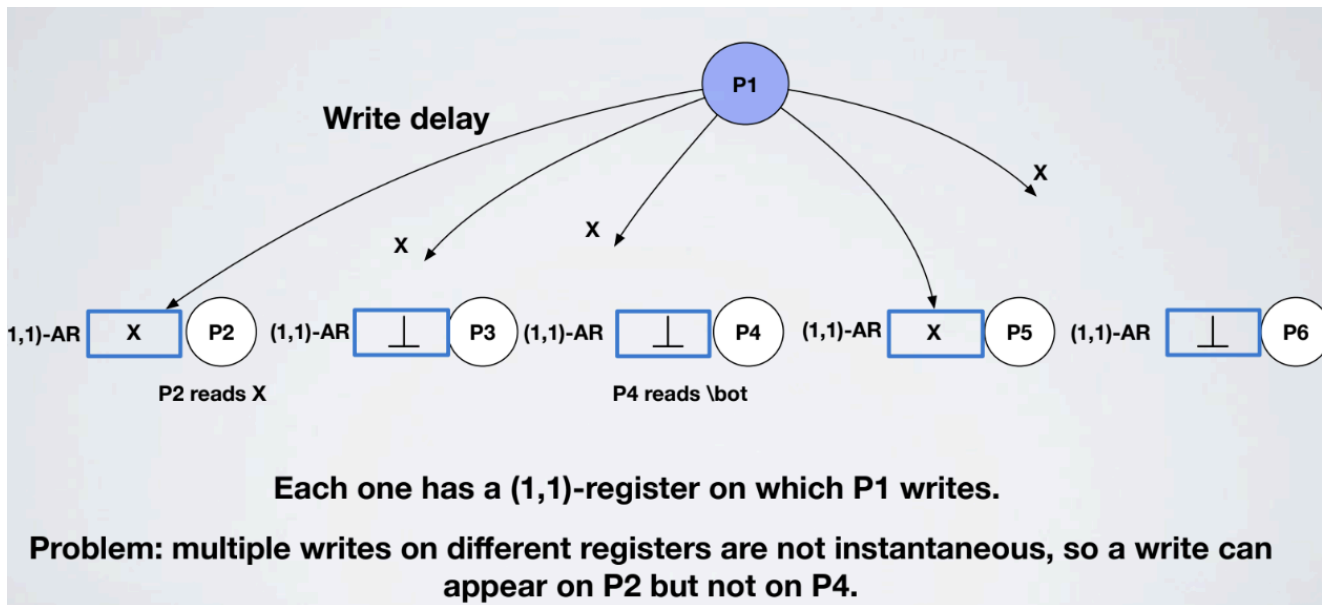
Correctness:

- **Termination** – from the termination property of the regular register
- **Validity** – from the validity property of the regular register
- **Ordering** – from the validity property and from the fact that the read tracks the last value read and its timestamp. A read operation always returns a value with a timestamp greater or equal to the one of the previously read value

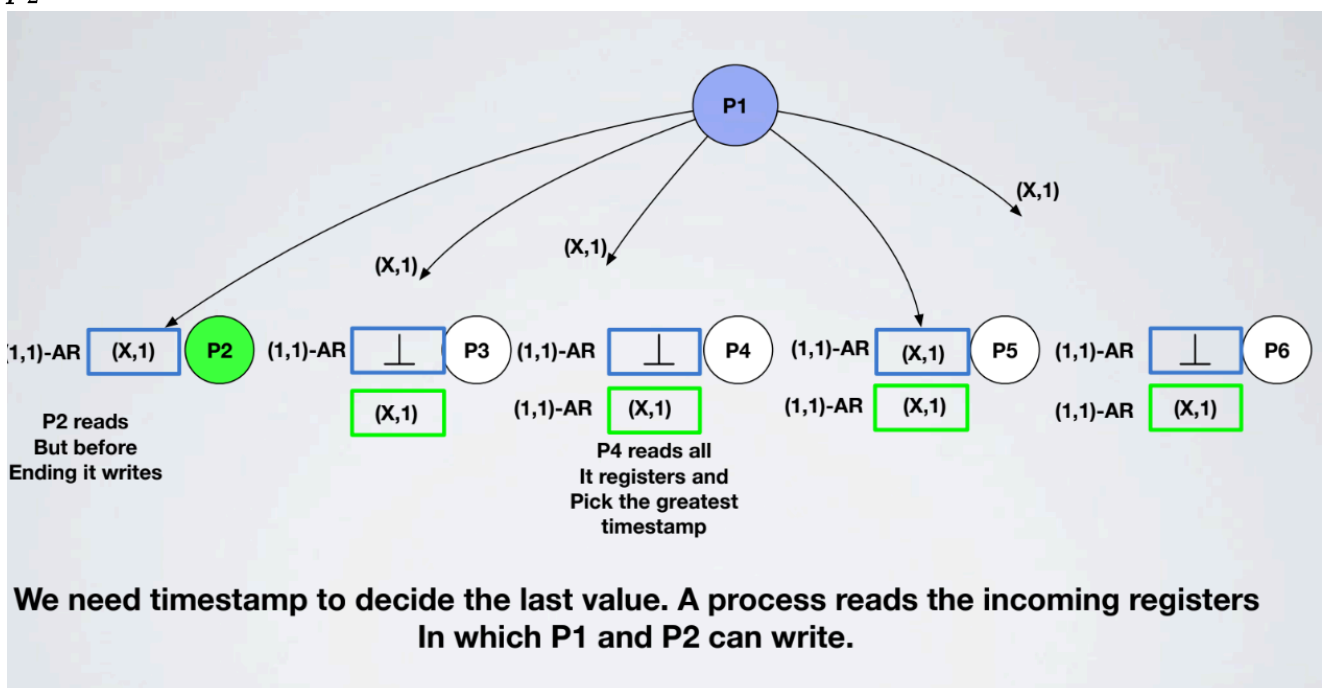
- **Write** – Each write operation performs a write on a (1,N) regular register
- **Read** - Each read operation performs a read on a (1,N) regular register

Let us say that we have p_1, p_2, p_3 , with p_1 able to write and everyone able to read. When p_1 wants to write, he will write on every $(1, 1)$ register implemented between the writer and each reader.

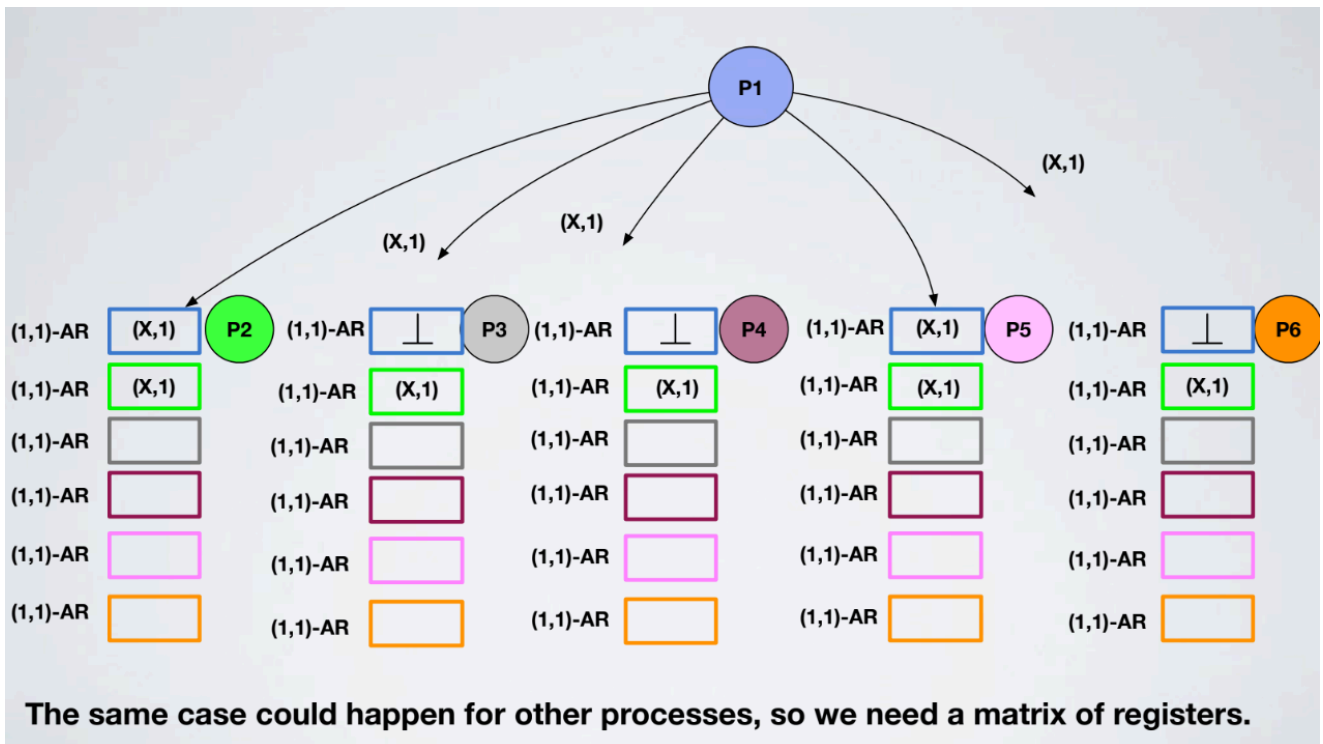
We'll have something like this:



The problem is that a write operation is not instantaneous, it may take some time. There is then a write delay different for each register, the time p_1 takes to complete a write on the register of p_2 , could be less than the one for p_3 (example). So, let's say p_2 has received x , and to read the value of the register he will read the local copy. p_4 , in the other hand, will see $\perp$, as he has not received yet x . How to avoid this? We'll need more $(1,1)$ registers between each couple of readers. With these registers we are simulating a channel. We'll use timestamps as well, as p_2 has to know what is the new value.



As we have to have a $(1,1)$ register between each couple, we'll have N^2 registers.



For example, on the grey line only p_3 has the permission to write.

In this algorithm, the reader helps the writer to complete the write.

Let's see the code:

upon event $\langle onar, Init \rangle$ **do**

$ts := 0;$

$acks := 0;$

$writing := FALSE;$

$readval := \perp;$

$readlist := [\perp]^N;$

forall $q \in \Pi, r \in \Pi$ **do**

Initialize a new instance $ooar.q.r$ of $(1, 1)$ -AtomicRegister
with writer r and reader q ;

OOAR.X.Y is a single register in a matrix
X is the reader
And Y is the writer

The acks will be used to track the return of the writes. Writing set to false is used to distinguish if I'm the original or the reader. readval is used by the reader for the value he has to read from the local buffer. readlist contains the scan of all the values of the readers.

Then, we have an initialization for all couple of processes in the system, initializing a $(1, 1)$ atomic register.

PHASE 2

```

upon event  $\langle onar, Read \rangle$  do
    forall  $r \in \Pi$  do
        trigger  $\langle ooar.self.r, Read \rangle$ ;

upon event  $\langle ooar.self.r, ReadReturn \mid (ts', v') \rangle$  do
     $readlist[r] := (ts', v')$ ;
    if  $\#(readlist) = N$  then
         $(maxts, readval) := \text{highest}(readlist)$ ;
         $readlist := [\perp]^N$ ;
        forall  $q \in \Pi$  do
            trigger  $\langle ooar.q.self, Write \mid (maxts, readval) \rangle$ ;

upon event  $\langle onar, Write \mid v \rangle$  do
     $ts := ts + 1$ ;
     $writing := \text{TRUE}$ ;
    forall  $q \in \Pi$  do
        trigger  $\langle ooar.q.self, Write \mid (ts, v) \rangle$ ;

upon event  $\langle ooar.q.self, WriteReturn \rangle$  do
     $acks := acks + 1$ ;
    if  $acks = N$  then
         $acks := 0$ ;
        if  $writing = \text{TRUE}$  then
            trigger  $\langle onar, WriteReturn \rangle$ ;
             $writing := \text{FALSE}$ ;
        else
            trigger  $\langle onar, ReadReturn \mid readval \rangle$ ;

```

When the writer writes, he increase the timestamp, he set writing to true, and trigger the writing to each couple of $(p_i, self)$. Only the writer process can handle this handler. Others can only trigger the read.

When a register returns, we increase the variable ACK. When we have the value of ACK updated to N , we set it to zero, and I was the original writer, I return the confirmation for the atomic write, and set writing to false.

When I want to read, I check all registers (my column).

When the reads start to terminate, I write in the corresponding location the value. I do something when the number of locations in readlist different from bottom is N . Then, I take the element with the highest timestamp, I reset the readlist, and for each register I write this new value.

Then the reader will see a writerreturn, receiving acks. When acks reaches N , i execute the "else" condition, which means that I was able to propagate the value, and thus terminate.

A write on the atomic register, how many on the $(1, 1)$ registers triggers? N , as it has to write on each register. How many operations is a read from the atomic registers? N reads on the $(1, 1)$, and N writes on the $(1, 1)$, so we have $2N$ operations. A read costs more than a write.

Let's see the proof:

Termination – from the termination of the (1,1) atomic register

Validity – from the validity of the (1,1) atomic register.

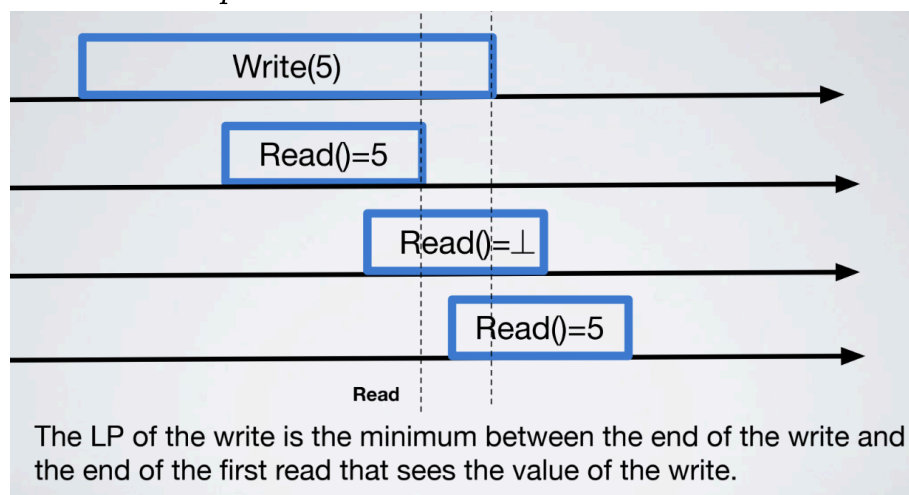
Ordering - Consider a write operation w_1 which writes value v_1 with timestamp s_1 . Let w_2 be a write that starts after w_1 . Let v_2 and s_2 ($s_1 < s_2$) be the value and the timestamp corresponding to w_2 .

Let assume that a read returns v_2 : by the algorithm, for each j in $[1;N]$, p_i has written (s_2, v_2) in $\text{readers}[r; i; j]$.

For the ordering property of the underlying (1,1) atomic registers, each successive read will return a value with timestamp greater or equal to s_2 . Then s_1 cannot be returned.

Where is the linearization point of this algorithm?

i missed this part



What is the resilience of the algorithm? If a process fails, because of termination property, even if we read from a determined location of a failed process, it's not a problem, as it will not block us. So, how many failures can we tolerate? $N - 1$.

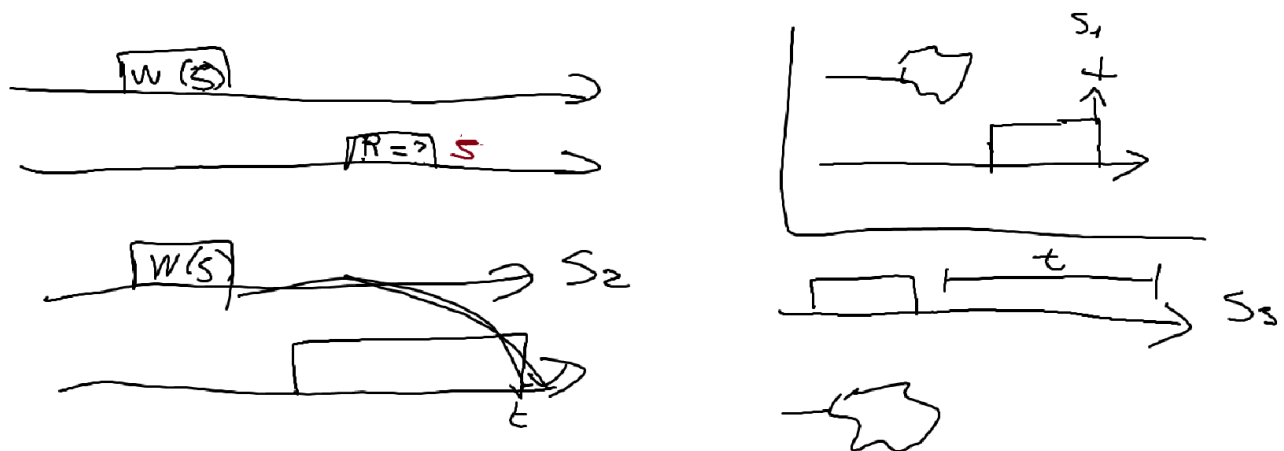
Let's now see an exercise:

Exercise 4.8: Does any implementation of a regular register require a majority of the correct processes in a fail-silent model with no failure detector? What if an eventually perfect failure detector (Module 2.8) is available?

How do we implement the RR in async systems? We do it with quorums. To use them, we have to have that $f < \frac{n}{2}$ ($c \geq \frac{n}{2} + 1$).

So, the exercise is asking if we can implement RR if $f = \frac{n}{2}$.

No. Let us imagine we are in a system where half of the processes can die. Let's now take a subsystem of two processes, in that case we can have 1 failure. Let's see the example in photo:



In the subsystem s_1 the first process crashes, and then the second process has to return, at time t , a value, which is $\perp$.

Let's also take s_2 . The first process writes 5, then decide to send all messages of writing after time t (we can decide as we are in async). The reader has to terminate at time t .

At time t I will terminate and return $\perp$. However, by returning this value, I should have returned 5 (but the write message containing 5 arrives after time t).

We can essentially exploit the indistinguishability to create the same systems.

This kind of technique used is known as **partition argument**. We can use this technique in asynchronous to create subsystems in which processes only communicates between themselves.

We can only use it in async systems.

In a system with quorums, a single "guy" cannot decide by itself, there has to be a process p_3 confirming/denying.

check the slides for better understanding.